Instagram Enginee... · Follow publication

What Powers Instagram: Hundreds of Instances, Dozens of Technologies

Instagram

6 min read · Dec 2, 2011

Instagram Engineering Follow

Listen Share ••• More

One of the questions we always get asked at meet-ups and conversations with other engineers is, “what’s your stack?” We thought it would be fun to give a sense of all the systems that power Instagram, at a high-level; you can look forward to more in-depth descriptions of some of these systems in the future. This is how our system has evolved in the just-over-1-year that we’ve been live, and while there are parts we’re always re-working, this is a glimpse of how a startup with a small engineering team can scale to our 14 million+ users in a little over a year. Our core principles when choosing a system are:

在技术交流会和与其他工程师的对话中，我们最常被问到的问题就是："你们的技术栈是什么？"我们认为，从宏观层面介绍支撑 Instagram 运行的所有系统会很有趣；未来你们还可以期待关于其中某些系统的深入解析。以下是我们上线仅一年多的系统演进历程，虽然某些部分我们仍在持续优化，但这展现了一个小型工程团队如何在短短一年多时间里将初创公司扩展至 1400 万+用户规模。我们选择系统时的核心原则是：

- Keep it very simple 保持极简

- Don’t re-invent the wheel

  避免重复造轮子

- Go with proven and solid technologies when you can

  尽可能选择经过验证的可靠技术

We’ll go from top to bottom:

我们将从上至下进行介绍：

OS / Hosting /

We run Ubuntu Linux 11.04 (“Natty Narwhal”) on Amazon EC2. We’ve found previous versions of Ubuntu had all sorts of unpredictable freezing episodes on EC2 under high traffic, but Natty has been solid. We’ve only got 3 engineers, and our needs are still evolving, so self-hosting isn’t an option we’ve explored too deeply yet, though is something we may revisit in the future given the unparalleled growth in usage.

我们在亚马逊 EC2 上运行 Ubuntu Linux 11.04（"Natty Narwhal"）。我们发现早期版本的 Ubuntu 在高流量下会出现各种不可预测的卡死问题，但 Natty 版本表现稳定。目前我们只有 3 名工程师，需求仍在不断变化，因此尚未深入探索自建服务器的方案——不过考虑到用户量呈指数级增长，未来可能会重新考虑这个选项。

Load Balancing

Every request to Instagram servers goes through load balancing machines; we used to run 2 nginx machines and DNS Round-Robin between them. The downside of this approach is the time it takes for DNS to update in case one of the machines needs to get decomissioned. Recently, we moved to using Amazon’s Elastic Load Balancer, with 3 NGINX instances behind it that can be swapped in and out (and are automatically taken out of rotation if they fail a health check). We also terminate our SSL at the ELB level, which lessens the CPU load on nginx. We use Amazon’s Route53

for DNS, which they've recently added a pretty good GUI tool for in the AWS console.

每一条发往 Instagram 服务器的请求都会经过负载均衡机器；我们过去使用 2 台 nginx 机器并通过 DNS 轮询机制分配流量。这种方式的缺点在于当某台机器需要下线时，DNS 更新所需的时间较长。近期我们已改用亚马逊弹性负载均衡器（ELB），在其后方配置了 3 个可随时替换的 NGINX 实例（若健康检查失败会自动移出服务集群）。我们还在 ELB 层级完成 SSL 终止，这减轻了 nginx 的 CPU 负载。DNS 解析采用亚马逊 Route53 服务，该服务近期在 AWS 控制台新增了相当完善的图形化管理工具。

Application Servers

Next up comes the application servers that handle our requests. We run Djangoon Amazon High-CPU Extra-Large machines, and as our usage grows we've gone from just a few of these machines to over 25 of them (luckily, this is one area that's easy to horizontally scale as they are stateless). We've found that our particular work-load is very CPU-bound rather than memory-bound, so the High-CPU Extra-Large instance type provides the right balance of memory and CPU.

下一层是处理请求的应用服务器。我们在亚马逊高 CPU 超大机型上运行 Django，随着业务增长，这类服务器已从最初的几台扩展到超过 25 台（幸运的是，由于采用无状态架构，这个层面很容易实现水平扩展）。我们发现业务负载主要消耗 CPU 资源而非内存资源，因此高 CPU 超大实例类型能提供理想的内存与 CPU 配比。

We use http://gunicorn.org/ as our WSGI server; we used to use mod_wsgi and Apache, but found Gunicorn was much easier to configure, and less CPU-intensive. To run commands on many instances at once (like deploying code), we use Fabric, which recently added a useful parallel mode so that deploys take a matter of seconds.

我们采用 http://gunicorn.org/作为 WSGI 服务器；过去使用 mod_wsgi 和 Apache 时，发现 Gunicorn 配置更简便且 CPU 负载更低。通过 Fabric 工具批量执行多实例命令（如代码部署），其新增的并行模式使部署过程仅需数秒。

Data storage

Most of our data (users, photo metadata, tags, etc) lives in PostgreSQL; we've previously written about how we shard across our different Postgres instances. Our main shard cluster involves 12 Quadruple Extra-Large memory instances (and

twelve replicas in a different zone.)

大部分数据（用户信息、照片元数据、标签等）存储在 PostgreSQL 中，我们曾撰文介绍过跨 Postgres 实例的分片方案。主分片集群包含 12 台四倍超大内存实例（以及位于不同可用区的 12 个副本）。

We've found that Amazon's network disk system (EBS) doesn't support enough disk seeks per second, so having all of our working set in memory is extremely important. To get reasonable IO performance, we set up our EBS drives in a software RAID using mdadm.

由于亚马逊网络磁盘系统（EBS）的每秒磁盘寻道能力不足，我们将工作数据集全部保留在内存中至关重要。为获得合理的 IO 性能，我们使用 mdadm 将 EBS 驱动器配置为软件 RAID 阵列。

As a quick tip, we've found that vmtouch is a fantastic tool for managing what data is in memory, especially when failing over from one machine to another where there is no active memory profile already. Here is the script we use to parse the output of a vmtouch run on one machine and print out the corresponding vmtouch command to run on another system to match its current memory status.

快速提示：我们发现 vmtouch 是管理内存数据的绝佳工具，特别是在从一台机器故障转移到另一台尚未建立活动内存配置的机器时。这是我们用来解析某台机器上 vmtouch 运行输出，并打印出对应 vmtouch 命令的脚本，以便在另一台系统上运行以匹配其当前内存状态。

All of our PostgreSQL instances run in a master-replica setup using Streaming Replication, and we use EBS snapshotting to take frequent backups of our systems. We use XFS as our file system, which lets us freeze & unfreeze the RAID arrays when snapshotting, in order to guarantee a consistent snapshot (our original inspiration came from ec2-consistent-snapshot. To get streaming replication started, our favorite tool is repmgr by the folks at 2ndQuadrant.

我们所有的 PostgreSQL 实例都采用主从复制架构运行，使用流复制技术，并通过 EBS 快照功能对系统进行频繁备份。我们选用 XFS 作为文件系统，这样在创建快照时可以冻结和解冻 RAID 阵列，从而确保获得一致性快照（最初灵感来源于 ec2-consistent-snapshot）。在搭建流复制环境时，我们最青睐的工具是 2ndQuadrant 团队开发的 repmgr。

To connect to our databases from our app servers, we made early on that had a huge impact on performance was using Pgbouncer to pool our connections to PostgreSQL. We found Christophe Pettus's blog to be a great resource for Django, PostgreSQL and Pgbouncer tips.

在应用服务器连接数据库方面，我们早期采用的一个对性能产生重大影响的方案是使用 Pgbouncer 来管理 PostgreSQL 连接池。我们发现 Christophe Pettus 的博客是获取 Django、PostgreSQL 和 Pgbouncer 技巧的宝贵资源。

The photos themselves go straight to Amazon S3, which currently stores several terabytes of photo data for us. We use Amazon CloudFront as our CDN, which helps with image load times from users around the world (like in Japan, our second most-popular country).

照片数据直接存储于亚马逊 S3 服务，目前为我们保存着数 TB 的图片资料。我们采用亚马逊 CloudFront 作为内容分发网络(CDN)，显著提升了全球用户（例如我们的第二大用户国日本）的图片加载速度。

We also use Redis extensively; it powers our main feed, our activity feed, our sessions system (here's our Django session backend), and other related systems. All of Redis' data needs to fit in memory, so we end up running several Quadruple Extra-Large Memory instances for Redis, too, and occasionally shard across a few Redis instances for any given subsystem. We run Redis in a master-replica setup, and have the replicas constantly saving the DB out to disk, and finally use EBS snapshots to backup those DB dumps (we found that dumping the DB on the master was too taxing). Since Redis allows writes to its replicas, it makes for very easy online failover to a new Redis machine, without requiring any downtime.

我们还大量运用 Redis 技术：它支撑着我们的主信息流、动态消息系统、会话管理系统（这是我们基于 Django 开发的会话后端）以及其他相关系统。由于 Redis 所有数据都需常驻内存，我们为此专门配置了多台四倍超大内存实例来运行 Redis 服务，并针对特定子系统偶尔会采用跨多个 Redis 实例的分片方案。我们采用主从架构部署 Redis，从节点会持续将数据库转储到磁盘，最终通过 EBS 快照备份这些转储文件（实践中发现主节点执行转储操作会消耗过多资源）。得益于 Redis 允许向从节点写入的特性，我们能够轻松实现零停机切换至新 Redis 服务器的在线故障转移。

For our geo-search API, we used PostgreSQL for many months, but once our Media entries were sharded, moved over to using Apache Solr. It has a simple JSON

interface, so as far as our application is concerned, it's just another API to consume.

我们的地理位置搜索 API 曾长期使用 PostgreSQL 数据库，但在媒体数据分片后，我们转而采用 Apache Solr 解决方案。该平台提供简洁的 JSON 接口，从应用层视角来看，这不过是又一个待调用的 API 服务。

Finally, like any modern Web service, we use Memcached for caching, and currently have 6 Memcached instances, which we connect to using pylibmc & libmemcached. Amazon has an Elastic Cache service they've recently launched, but it's not any cheaper than running our instances, so we haven't pushed ourselves to switch quite yet.

与所有现代化网络服务相同，我们采用 Memcached 实现缓存功能，目前通过 pylibmc 和 libmemcached 连接着 6 个 Memcached 实例。亚马逊虽已推出 Elastic Cache 缓存服务，但其成本并不比自主运维更具优势，因此我们暂未进行服务迁移。

Task Queue & Push Notifications

When a user decides to share out an Instagram photo to Twitter or Facebook, or when we need to notify one of our Real-time subscribers of a new photo posted, we push that task into Gearman, a task queue system originally written at Danga. Doing it asynchronously through the task queue means that media uploads can finish quickly, while the 'heavy lifting' can run in the background. We have about 200 workers (all written in Python) consuming the task queue at any given time, split between the services we share to. We also do our feed fan-out in Gearman, so posting is as responsive for a new user as it is for a user with many followers.

当用户决定将 Instagram 照片分享到 Twitter 或 Facebook 时，或者当我们需要向实时订阅者通知新发布的照片时，我们会将该任务推送到 Gearman——这个任务队列系统最初由 Danga 编写。通过任务队列异步处理意味着媒体上传可以快速完成，而繁重的任务可以在后台运行。我们约有 200 个工作进程（全部用 Python 编写）随时处理任务队列，并根据需要分发给不同的分享服务。我们还在 Gearman 中完成动态推送，因此无论新用户还是拥有大量粉丝的用户，发布体验都同样流畅。

For doing push notifications, the most cost-effective solution we found was https://github.com/samuraisam/pyapns, an open-source Twisted service that has

handled over a billion push notifications for us, and has been rock-solid.

在推送通知方面，我们发现最具成本效益的解决方案是 https://github.com/samuraisam/pyapns，这个开源的 Twisted 服务已为我们处理了超过 10 亿条推送通知，并且运行极其稳定。

Monitoring

With 100+ instances, it's important to keep on top of what's going on across the board. We use Munin to graph metrics across all of our system, and also alert us if anything is outside of its normal range. We write a lot of custom Munin plugins, building on top of Python-Munin, to graph metrics that aren't system-level (for example, signups per minute, photos posted per second, etc). We use Pingdom for external monitoring of the service, and PagerDuty for handling notifications and incidents.

拥有 100 多个实例时，全面掌握系统运行状况至关重要。我们使用 Munin 绘制全系统指标图表，并在数值超出正常范围时发出警报。基于 Python-Munin 框架，我们开发了大量自定义插件来监控非系统级指标（例如每分钟注册量、每秒照片发布量等）。Pingdom 负责服务的外部监控，PagerDuty 则处理通知和事件响应。

For Python error reporting, we use Sentry, an awesome open-source Django app written by the folks at Disqus. At any given time, we can sign-on and see what errors are happening across our system, in real time.

针对 Python 错误报告，我们采用 Disqus 团队开发的开源 Django 应用 Sentry。通过它，我们可以随时登录系统实时查看各处发生的错误。

You?

If this description of our systems interests you, or if you're hopping up and down ready to tell us all the things you'd change in the system, we'd love to hear from you. We're looking for a DevOps person to join us and help us tame our EC2 instance herd.

如果我们的系统架构让你感兴趣，或者你已经迫不及待想告诉我们你会如何改进这套系统，我们期待你的联系。目前正在招募 DevOps 工程师加入团队，共同管理日益壮大的 EC2 实例集群。

Cloud Computing AWS Instagram